

Network Packet Sniffing and Credential Analysis Using Wireshark

EXPERIMENT NO: 03

DATE:

1. AIM

To capture and analyze unencrypted network traffic using the Wireshark packet analyzer, isolate HTTP GET and POST request methods, and extract transmitted plain-text login credentials (username and password).

2. SOFTWARE / TOOLS REQUIRED

- **Operating System:** Windows 10/11 or Kali Linux
- **Packet Analyzer:** Wireshark v4.x (with Npcap driver installed)
- **Web Browser:** Google Chrome / Mozilla Firefox
- **Target Environment:** Acunetix Web Vulnerability Test Site
([<http://testphp.vulnweb.com>])(<http://testphp.vulnweb.com>)

3. DESCRIPTION

Network packet sniffing is the process of monitoring and capturing data packets passing through a network interface card (NIC). Wireshark utilizes packet capture drivers (Npcap/libpcap) to set the system's NIC into **promiscuous mode**, allowing it to capture all frames traversing the local network segment.

When web applications transmit data using unencrypted protocols such as **HTTP (Hypertext Transfer Protocol)** over TCP port 80, the payload remains in ASCII plain text:

- **HTTP GET Method:** Appends parameters to the URL string. While used primarily for retrieving data, it leaves information visible in server logs and browser histories.
- **HTTP POST Method:** Embeds data within the body of the request header (Content-Type: application/x-www-form-urlencoded). Although hidden from the URL bar, the payload remains completely legible to network sniffers if not protected by TLS/SSL encryption (**HTTPS**).

4. PROCEDURE

Step 1: Interface Initialization

1. Launch Wireshark.
2. Select the active **Wi-Fi** network interface to begin live traffic capture.

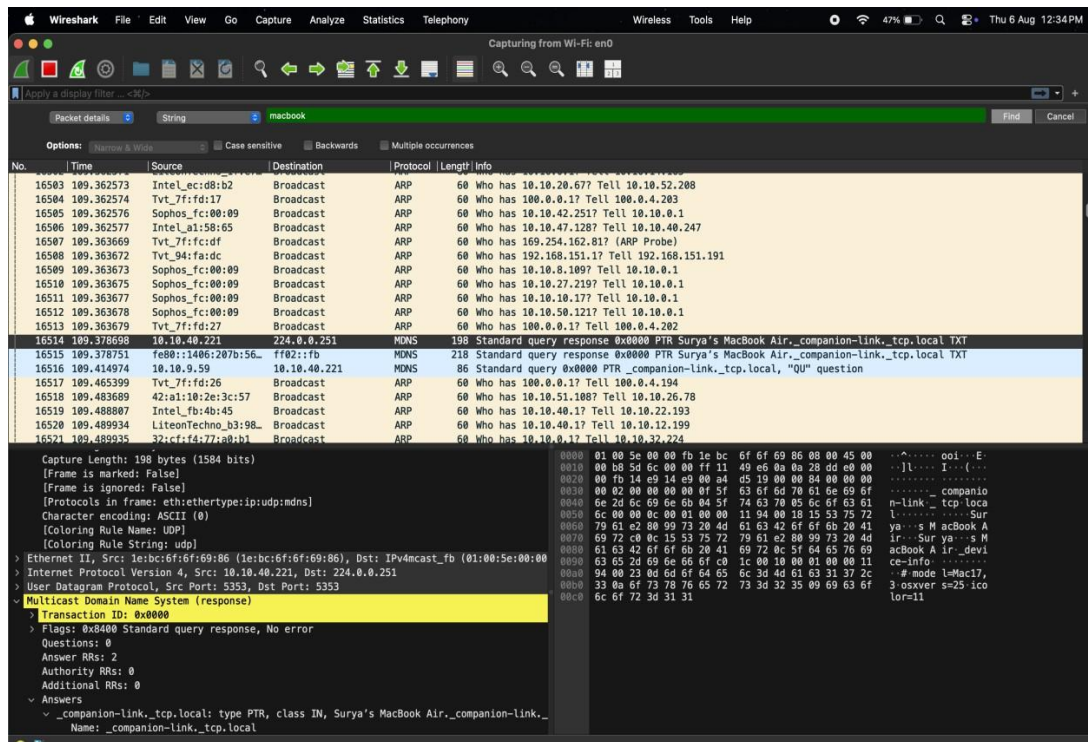


Figure 1: Wireshark Interface Capture Selection

Step 2: Generating Network Traffic

1. Open the target web page in a browser.
2. Enter the credential details (Username: macbook, Password: macbokk m5) into the form fields and submit.

Step 4: Inspecting HTTP GET Requests

Enter the following filter to inspect GET requests:

```
http.request.method == "GET"
```

Observation: GET packets retrieve page components (/login.aspx), but do not carry submitted form parameters in their payload.

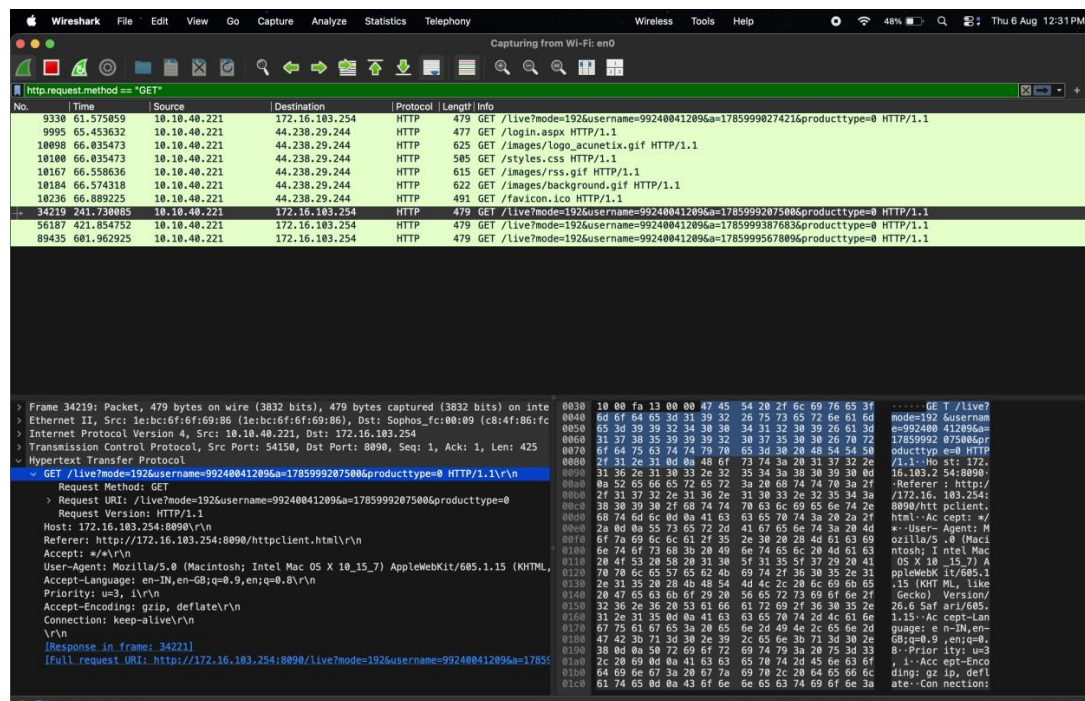


Figure 4: Analysis of Captured HTTP GET Requests

Step 5: Extracting POST Payload and Credentials

Apply the POST filter to locate form submission requests:

```
http.request.method == "POST"
```

Select the packet containing application/x-www-form-urlencoded. In the **Packet Details** pane, expand the **HTML Form URL Encoded** tree to read the plain-text form variables.

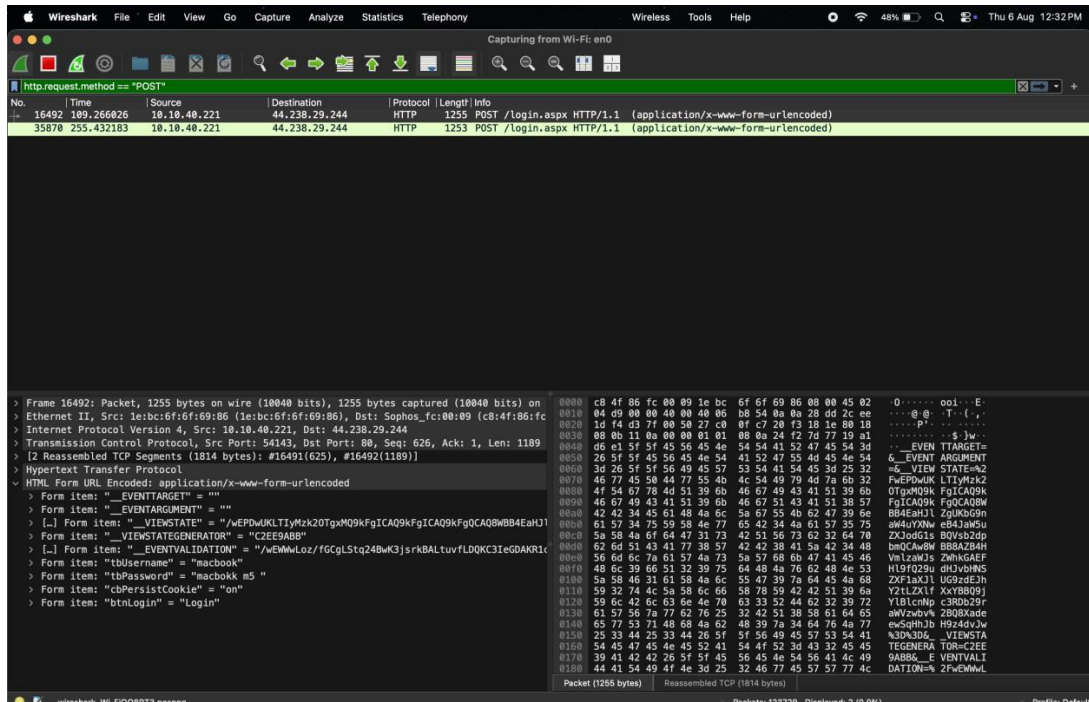


Figure 5: Inspecting HTTP POST Packet Payload

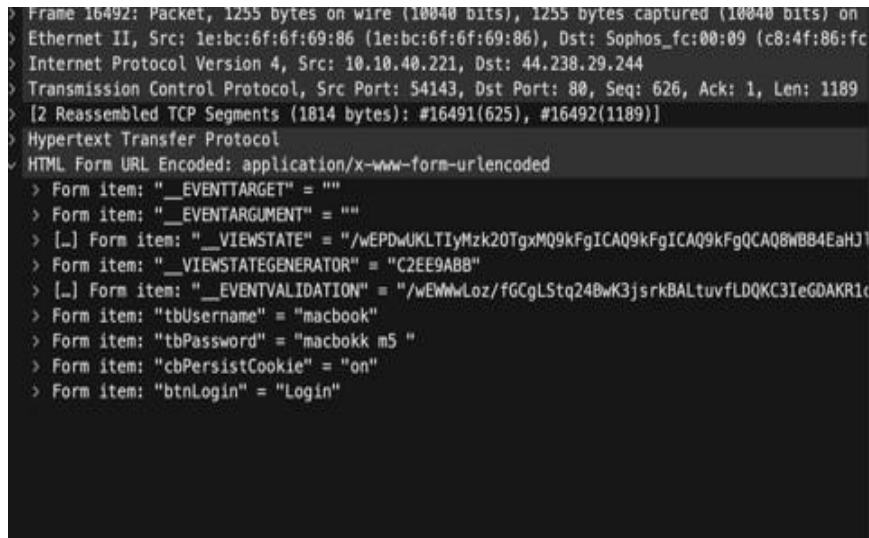


Figure 6: Plain-text Credential Extraction from Form Fields

5. OBSERVATIONS

Extracted Form Parameters

Field Name	Captured Key	Extracted Value
Username	tbUsername	macbook
Password	pass / tbPassword	Macbokk m5

6. VERIFICATION OF SAVED LOGS & DIRECTORY STRUCTURE

The packet capture data (Captured_File.pcapng) and generated screen outputs have been archived in the repository.

7. PREVENTIVE COUNTERMEASURES

1. **Enforce HTTPS (TLS 1.3):** Encrypt all client-server communications so intercepted payloads appear as encrypted ciphertext.
2. **HTTP Strict Transport Security (HSTS):** Enforce browser-side HSTS policies to prevent downgrade attacks to plain HTTP.
3. **Network Isolation:** Implement VLANs and 802.1X port-based access controls to restrict unauthorized packet sniffing on local physical and wireless networks.

8. RESULT

The network traffic was successfully captured and analyzed using Wireshark. Plain-text credentials (**Username:** macbook, **Password:** macbokk m5) transmitted over HTTP were extracted from the POST request payload.